

03 — Example 1: MO-book production planning, from equations to Python objects

03-E01.1 Purpose and files

This learning report connects a small mathematical model to the Python objects discussed in Chapter 03. Follow one quantity from an input `float`, through a symbolic optimization expression, to a solved number and a labeled pandas table. The goal is to explain what the code means, not just obtain a solver answer.

The example adapts MO-book §1.2, with the problem introduced in §1.1. It is a one-week, single-solve linear program (LP), using textbook data. It is neither a mixed-integer linear program (MILP) nor rolling-horizon planning, and its profit is not company performance. The adaptation preserves the five-variable model while adding numbered Spyder cells, editable inputs, result tables, and checks.

Use the standalone [Python script](#). Its upstream copyright and MIT license are retained in [LICENSE-MO-book.txt](#). The script computes in memory and prints results; it does not read business inputs, write output files, or use a network connection.

03-E01.2 Inputs, decisions, and units

Two products, U and V, consume material M and two kinds of labor, A and B. Choose production and the resources purchased or allocated for this week. The U sales ceiling is not a requirement to make 40 units. V has no separate sales ceiling, but labor still limits production. Money is expressed in generic monetary units. [Problem source: MO-book §1.1](#) ^[1].

Input per product unit	U	V
Selling price, monetary units/unit	270	210
Material M, grams/unit	10	9
Labor A, hours/unit	1	1
Labor B, hours/unit	2	1
Sales ceiling, units/week	40	Not specified

Resource	Unit cost	Weekly availability
Material M	10 monetary units/gram	No explicit ceiling
Labor A	50 monetary units/hour	80 hours
Labor B	40 monetary units/hour	100 hours

All five decisions are nonnegative and continuous. A numerical answer that happens to be an integer does not mean integer restrictions were imposed. There is no inventory carried between weeks.

Mathematical symbol	Code object	Meaning and unit
y_U	<code>model.y_U</code>	U output, units/week
y_V	<code>model.y_V</code>	V output, units/week
x_M	<code>model.x_M</code>	Material allocation, grams/week
x_A	<code>model.x_A</code>	Labor A allocation, hours/week
x_B	<code>model.x_B</code>	Labor B allocation, hours/week

The letters A and B name labor categories, not additional products. Check dimensions before calculating: monetary units/gram multiplied by grams/week gives monetary units/week.

03-E01.3 The mathematical model and its implementation

Maximize weekly revenue minus material and labor costs:

(03-E01.1) maximize $P = 270y_U + 210y_V - 10x_M - 50x_A - 40x_B$.

Production cannot consume more resources than allocated:

(03-E01.2) $10y_U + 9y_V \leq x_M$.

(03-E01.3) $y_U + y_V \leq x_A$.

(03-E01.4) $2y_U + y_V \leq x_B$.

The variable domains and upper bounds complete the model:

(03-E01.5) $x_M \geq 0$; $0 \leq x_A \leq 80$; $0 \leq x_B \leq 100$; $0 \leq y_U \leq 40$; $y_V \geq 0$.

In S3, `pyo.NonNegativeReals` supplies nonnegativity and continuity; `bounds=(0, labor_b_limit)` supplies B's capacity. In S4, `model.revenue` and `model.cost` are expressions, not two extra decision variables. `model.profit` is the objective with `sense=pyo.maximize`. In S5, the following exact script fragment encodes equation (03-E01.4):

```
model.labor_B = pyo.Constraint(
    expr=labor_b_per_u * model.y_U + labor_b_per_v * model.y_V <= model.x_B
)
```

Here `<=` creates a symbolic relation for Pyomo; it is not a solved yes/no result. The resource coefficients map directly to `material_per_u`, `material_per_v`, `labor_a_per_u`, `labor_a_per_v`, `labor_b_per_u`, and `labor_b_per_v`. Prices map to `price_u` and `price_v`; resource cost rates map to `material_cost`, `labor_a_cost`, and `labor_b_cost`.

Every resource has a positive cost. At an optimum, allocating more than production requires would waste money, so the three resource inequalities are tight. This observation permits the two-output hand calculation later; the runnable script still retains all five variables.

03-E01.4 Read the script by stable cell number

The # %% markers divide the file into Spyder cells. The cell identifiers stay meaningful even if line numbers change. Read the mathematical model alongside S2–S5, then trace the conversion into ordinary results in S6–S8.

Cell	Role	What to inspect
E01-S0	Scope and source	Textbook LP; adaptation notice
E01-S1	Import libraries	pyo for modeling; pd for tables
E01-S2	Set numerical inputs	Prices, costs, coefficients, capacities
E01-S3	Create model and five variables	Domains, bounds, unknown values
E01-S4	Define expressions and objective	Revenue, cost, maximize profit
E01-S5	Add resource constraints	Consumption versus allocation
E01-S6	Solve with HiGHS	Availability and optimal termination
E01-S7	Extract numbers and build tables	production, resources, financials
E01-S8	Recompute checks and print	checks, tolerance, displayed results

S6 first solves with `load_solutions=False`, then requires `TerminationCondition.optimal` before loading values. This avoids presenting unaccepted solver output as a valid solution. A missing solver or nonoptimal termination raises an error instead of silently continuing.

03-E01.5 Python parameters are not optimization variables

The word “variable” has two meanings here. Python names reference objects; an optimization decision variable represents an unknown quantity in the model. A known price may be stored under a Python name without becoming a decision for the solver.

Example	Kind of object	Interpretation
<code>price_u = 270.0</code>	Python float	Known numerical input
<code>bounds=(0, labor_a_limit)</code>	Python tuple	Pair of lower and upper bounds
<code>model.y_U</code>	Pyomo variable component	Symbolic unknown before solving
<code>model.revenue</code>	Pyomo expression component	Formula referencing decisions
<code>u_units = pyo.value(model.y_U)</code>	Solved numeric value; float in the recorded run	Snapshot of the loaded solution
<code>termination</code>	Python str	Human-readable solver termination
<code>checks</code>	pandas Series of Boolean values	Named validation results

S2 uses ordinary Python floats, not `pyo.Param` components. When S3–S5 construct the model, those values enter its bounds and expressions. Reassigning `labor_b_limit = 120.0` later does not retroactively update the existing model. Rerun the whole file after input edits, including model creation and solving.

Before solving, `model.y_U.value` is `None: unknown`, not zero. After successful solution loading, `pyo.value(model.y_U)` extracts its numerical value. The name `u_units` holds that extracted value, not a live connection that updates automatically after another solve.

For inspection, run the following after the complete script. Type names for library components can be more specific than the broad categories above; that is normal.

```
type(price_u)
type(model.y_U)
type(u_units)
model.y_U.value
model.labor_B.pprint()
```

03-E01.6 From dictionary keys to pandas labels

In this exact S7 fragment, braces create a Python dictionary. Its string keys become the Series index labels; the numeric values become the Series contents. The `name` describes the whole Series, not another entry.

```
financials = pd.Series({
    "revenue": revenue_value, "cost": cost_value, "profit": profit_value,
}, name="monetary_units_per_week")
```

A Series is one-dimensional labeled data. `financials.loc["profit"]` selects by label; `financials.iloc[2]` selects the third position, counting from zero. They coincide here because of the construction order, but position is not the meaning of the label. Prefer the label when asking for profit. See the [Series API](#) ^[2] and [indexing guide](#) ^[3].

The second exact S7 fragment uses a dictionary differently: its keys become DataFrame column labels and its equal-length lists supply the columns.

```
production = pd.DataFrame({
    "product": ["U", "V"],
    "quantity_units_per_week": [u_units, v_units],
})
```

A DataFrame is two-dimensional. Here the default row index is 0, 1; U and V are values in the `product` column, not row labels. Thus `production.loc[0, "quantity_units_per_week"]` selects row label 0 and the named column, whereas `production.iloc[0, 1]` selects the first row and second column by position. `production["product"]` returns a Series. See the [DataFrame API](#) ^[4].

Python dictionary keys are unique: assigning the same key again replaces its associated value. pandas labels are not universally required to be unique. A dictionary's key rule therefore must not be mistaken for a general Series or DataFrame restriction. The containers are related by construction, not identical types. [Python dictionary reference](#) ^[5].

The `resources` DataFrame similarly uses the keys `resource`, `allocated`, `limit`, and `unit` as columns. Its `limit` list contains `[None, labor_a_limit, labor_b_limit]`; the material row displays `NaN` in this numeric column. Here that intentionally represents “no explicit material ceiling,” not zero capacity. This is a modeling convention, not a universal interpretation of missing data.

03-E01.7 Run and inspect in Spyder

Use the existing `Jushi_scheduler` environment with `pyomo`, `highspy`, and `pandas`. In an Anaconda Prompt, start from the handbook root:

```
conda activate Jushi_scheduler
python examples\03_example_01_mo_book_production.py
```

For Spyder, use this procedure:

1. Open the same script and use a console whose Python interpreter belongs to `Jushi_scheduler`. Check `import sys; print(sys.executable)` in that console if uncertain.
2. Run the entire file first. Read the termination message and confirm that the independent checks pass.
3. In Variable Explorer, open `production`, `resources`, `financials`, and `checks`. These ordinary tables are easier to inspect than the nested `model` object.
4. To study construction, restart the console and run cells in order from E01-S1 through E01-S8. Before S6, inspect `model.y_U.value`; after S7, inspect `u_units`.
5. After changing S2 inputs, run the entire file again. Do not mix tables from an old solution with a newly edited parameter.

The console can also run `model.pprint()` to display model structure. If HiGHS is unavailable or an import fails, check the active interpreter first; installing packages into a different environment will not repair this console. Spyder interface labels and shortcuts may vary, so use its `run-file` and `run-cell` commands. [Spyder editor documentation](#) ^[6].

03-E01.8 Results, feasibility, and an optimality certificate

For the baseline inputs, reproduce the following result:

Output	Value
U production, units/week	20
V production, units/week	60
Material, grams/week	740
Labor A, hours/week	80
Labor B, hours/week	100
Revenue, monetary units/week	18000
Cost, monetary units/week	15400
Profit, monetary units/week	2600

Hand checks make the answer interpretable: material is $10 \cdot 20 + 9 \cdot 60 = 740$; labor A is $20 + 60 = 80$; labor B is $2 \cdot 20 + 60 = 100$. Revenue is $270 \cdot 20 + 210 \cdot 60 = 18000$, and cost is $10 \cdot 740 + 50 \cdot 80 + 40 \cdot 100 = 15400$. U has 20 units of unused sales allowance; both labor capacities are fully used.

The eight S8 checks test nonnegativity, three upper bounds, three resource balances, and the profit identity, with $\text{tolerance} = 1\text{e-}6$ for floating-point noise. Passing these checks establishes the tested feasibility and consistency conditions; primal feasibility alone does not prove optimality. The solver termination check is separate.

There is also a short analytical certificate. Write $U = y_U$ and $V = y_V$. With no resource waste, each U contributes $270 - 100 - 50 - 80 = 40$, and each V contributes $210 - 90 - 50 - 40 = 30$. Any excess allocation only lowers profit. Therefore every feasible plan satisfies:

$$(03\text{-E01.6}) P \leq 40U + 30V = 20(U + V) + 10(2U + V) \leq 20 \times 80 + 10 \times 100 = 2600.$$

The reported plan is feasible and attains 2600, so no feasible plan can improve it. This is an optimality argument, not just substitution into constraints. It uses the baseline coefficients and must be reconsidered after changing inputs.

03-E01.9 Controlled experiments and next steps

Change one input at a time, rerun all cells, record the result, and restore the baseline before the next experiment.

1. Increase `labor_b_limit` from 100 to 120. Expect $U = 40$, $V = 40$, and profit = 2800. Explain why greater B capacity changes the product mix although A capacity stays at 80.
2. Set `labor_a_limit` to zero with other inputs restored. Expect zero production, resource allocation, and profit: both products require A labor.
3. Retrieve profit using `.loc`, and retrieve V output from the DataFrame. Explain why `.loc["V"]` does not work on production as constructed.
4. Explain why `type(model.y_U)` differs from `type(u_units)`, and why an integer-looking production result is still an LP solution.

The script does not implement integer batches, setup decisions, equipment assignment, due dates, inventory dynamics, or uncertainty. A next example could introduce explicit integer/binary decisions and inventory balances to build a MILP. A subsequent rolling-horizon example would repeatedly solve a model, implement only the current decisions, and transfer the resulting state to the next window. Those are proposed learning steps, not features already implemented here.

03-E01.10 Reproducibility and sources

The accompanying [verification record](#) identifies the tested script by a SHA-256 content hash and records the actual checks. The recorded environment is Python 3.12.13, Pyomo 6.10.1, highspy 1.15.1, and pandas 3.0.5 in `jushi_scheduler`, dated 2026-09-25. Consult that record when comparing results after edits; an old hash does not verify changed code.

The separate [verification script](#) can be run from the handbook root in the same environment:

```
python examples\verify_03_example_01.py
```

Five cases passed on 2026-09-25: the baseline, B capacity 120, A capacity zero, all nine cells executed in order in a shared namespace, and deliberately infeasible A capacity -1 . The last case raised an error before loading a solution: there were zero solution-load calls, and all five decision values remained unset. The negative-capacity case tests failure handling; it is not a sensible business scenario.

Command-line execution and shared-namespace cell execution are distinct from operating Spyder's graphical interface. No Spyder GUI run or performance benchmark is claimed. The small tests check this teaching artifact, not a factory scheduling system or complete business-input validation.

The problem and modeling starting point are [MO-book §1.1](#) ^[1] and [MO-book §1.2](#) ^[7]. Code attribution is retained under the [upstream MIT license](#) ^[8]. For the library concepts, use the linked Python and pandas references above and [Pyomo's modeling components documentation](#) ^[9]. The cell numbering, object tracing, result checks, and hand certificate in this report explain the local learning adaptation.

References

1. <https://mobook.github.io/MO-book/notebooks/01/01-production-planning.html>
2. <https://pandas.pydata.org/docs/reference/api/pandas.Series.html>
3. https://pandas.pydata.org/docs/user_guide/indexing.html
4. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>
5. <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
6. <https://docs.spyder-ide.org/current/panes/editor.html>
7. <https://mobook.github.io/MO-book/notebooks/01/02-production-planning-basic.html>
8. <https://github.com/mobook/MO-book/blob/main/LICENSE>
9. https://pyomo.readthedocs.io/en/stable/explanation/modeling/math_programming/index.html